DSA Practical Manual

PRACTICAL 1

Implement Stack and its Operations using Linear Data Structure

Problem Statement

Implement a **Stack using an array** as a linear data structure and perform the following operations:

- Creation
- Push
- Pop
- Traverse
- Peek
- Search

A stack follows the **LIFO (Last In, First Out)** principle.

Algorithm

A. Create Stack

1. Declare an array `stack[MAX]`.
2. Initialize `top = -1`.
3. The stack is initially empty.

B. Push

1. Check whether `top == MAX - 1`.
2. If yes, display **Stack Overflow**.
3. Otherwise, increment `top`.

4. Insert the element at `stack[top]`.

C. Pop

1. Check whether `top == -1`.
2. If yes, display **Stack Underflow**.
3. Otherwise, display `stack[top]`.
4. Decrement `top`.

D. Peek

1. Check whether the stack is empty.
2. If empty, display an appropriate message.
3. Otherwise, display `stack[top]`.

E. Traverse

1. Check whether the stack is empty.
2. If empty, display an appropriate message.
3. Otherwise, start from `top`.
4. Display all elements up to index `0`.

F. Search

1. Read the element to be searched.
2. Start from `top`.
3. Compare each stack element with the search element.
4. If found, display its position.
5. If the complete stack is checked and the element is not found, display **Element not found**.

Input and Expected Output Test Case

Input

Push: 10
Push: 20
Push: 30
Push: 40

Expected Output

Stack elements:
40 30 20 10

Peek element: 40

Search element: 20
Element found at position 2

Pop element: 40

Source Code

```
#include <iostream>
using namespace std;

#define MAX 100

int stackArr[MAX];
int top = -1;

void push(int value)
{
   if (top == MAX - 1)
   {
      cout << "Stack Overflow\n";
      return;
   }

   top++;
   stackArr[top] = value;
   cout << value << " pushed into stack.\n";
}

void pop()
{
   if (top == -1)
   {
      cout << "Stack Underflow\n";
      return;
   }

   cout << "Popped element: " << stackArr[top] << endl;
   top--;
```

```
}

void traverse()
{
    if (top == -1)
    {
        cout << "Stack is empty.\n";
        return;
    }

    cout << "Stack elements: ";

    for (int i = top; i >= 0; i--)
    {
        cout << stackArr[i] << " ";
    }

    cout << endl;
}

void peek()
{
    if (top == -1)
    {
        cout << "Stack is empty.\n";
        return;
    }

    cout << "Peek element: " << stackArr[top] << endl;
}

void search(int value)
{
    if (top == -1)
    {
        cout << "Stack is empty.\n";
        return;
    }

    for (int i = top; i >= 0; i--)
    {
        if (stackArr[i] == value)
        {
            cout << "Element found at position " << i + 1 << endl;
```

```
            return;
        }
    }

    cout << "Element not found.\n";
}

int main()
{
    push(10);
    push(20);
    push(30);
    push(40);

    cout << endl;

    traverse();

    peek();

    search(20);

    pop();

    traverse();

    return 0;
}
```

Actual Output

```
10 pushed into stack.
20 pushed into stack.
30 pushed into stack.
40 pushed into stack.

Stack elements: 40 30 20 10
Peek element: 40
Element found at position 2
Popped element: 40
Stack elements: 30 20 10
```

PRACTICAL 2

Implement Infix to Postfix Expression Conversion using Stack

Problem Statement

Implement a program to convert an **infix expression into postfix expression using Stack**.

Example:

Infix: A+B*C
Postfix: ABC*+

The conversion should follow operator precedence.

Operator Precedence

^ Highest
* / %
+ - Lowest

Algorithm

1. Create an empty stack for operators.
2. Scan the infix expression from left to right.
3. If the character is an operand, add it directly to the postfix expression.
4. If the character is `(`, push it onto the stack.
5. If the character is `)`, pop operators until `(` is found.
6. If the character is an operator:
    - Pop operators having higher or equal precedence.
    - Push the current operator.
7. After scanning the expression, pop all remaining operators.
8. The resulting expression is the postfix expression.

Input and Expected Output Test Case

Input

A+B*C-D

Expected Output

ABC*+D-

Source Code

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

char stackArr[MAX];
int top = -1;

void push(char ch)
{
   stackArr[++top] = ch;
}

char pop()
{
   return stackArr[top--];
}

char peek()
{
   return stackArr[top];
}

int precedence(char ch)
{
   if (ch == '^')
      return 3;

   if (ch == '*' || ch == '/' || ch == '%')
      return 2;

   if (ch == '+' || ch == '-')
      return 1;

   return 0;
```

```
}

bool isOperand(char ch)
{
    return ((ch >= 'A' && ch <= 'Z') ||
            (ch >= 'a' && ch <= 'z') ||
            (ch >= '0' && ch <= '9'));
}

string infixToPostfix(string infix)
{
    string postfix = "";

    for (int i = 0; i < infix.length(); i++)
    {
        char ch = infix[i];

        if (ch == ' ')
            continue;

        if (isOperand(ch))
        {
            postfix += ch;
        }
        else if (ch == '(')
        {
            push(ch);
        }
        else if (ch == ')')
        {
            while (top != -1 && peek() != '(')
            {
                postfix += pop();
            }

            if (top != -1)
                pop();
        }
        else
        {
            while (top != -1 &&
                   peek() != '(' &&
                   precedence(peek()) >= precedence(ch))
            {
```

```
            postfix += pop();
        }

        push(ch);
    }
  }

  while (top != -1)
  {
     postfix += pop();
  }

  return postfix;
}

int main()
{
  string infix;

  cout << "Enter infix expression: ";
  cin >> infix;

  string postfix = infixToPostfix(infix);

  cout << "Postfix expression: " << postfix << endl;

  return 0;
}
```

Actual Output

Enter infix expression: A+B*C-D
Postfix expression: ABC*+D-

Another Example

Input:
(A+B)*C

Output:
AB+C*

PRACTICAL 3

Implement Postfix Evaluation using Stack

Problem Statement

Implement a program to **evaluate a postfix expression using Stack**.

For example:

Postfix Expression: 23*54*+

Evaluation:

2 × 3 = 6
5 × 4 = 20
6 + 20 = 26

Therefore:

Result = 26

Algorithm

1. Create an empty stack.
2. Scan the postfix expression from left to right.
3. If the character is an operand, convert it into an integer and push it onto the stack.
4. If the character is an operator:
    - Pop the top element as `operand2`.
    - Pop the next element as `operand1`.
    - Perform `operand1 operator operand2`.
    - Push the result onto the stack.
5. Continue until the expression is completely scanned.
6. The final element in the stack is the result.

Input and Expected Output Test Case

Input

23*54*+

Expected Output

Postfix expression: 23*54*+
Result: 26

Source Code

```
#include <iostream>
#include <string>
using namespace std;

#define MAX 100

int stackArr[MAX];
int top = -1;

void push(int value)
{
   stackArr[++top] = value;
}

int pop()
{
   return stackArr[top--];
}

int evaluatePostfix(string expression)
{
   for (int i = 0; i < expression.length(); i++)
   {
      char ch = expression[i];

      if (ch >= '0' && ch <= '9')
      {
         push(ch - '0');
      }
      else
      {
         int operand2 = pop();
         int operand1 = pop();

         int result;
```

```
            if (ch == '+')
                result = operand1 + operand2;
            else if (ch == '-')
                result = operand1 - operand2;
            else if (ch == '*')
                result = operand1 * operand2;
            else if (ch == '/')
                result = operand1 / operand2;
            else if (ch == '%')
                result = operand1 % operand2;

            push(result);
        }
    }

    return pop();
}

int main()
{
    string expression;

    cout << "Enter postfix expression: ";
    cin >> expression;

    int result = evaluatePostfix(expression);

    cout << "Result: " << result << endl;

    return 0;
}
```

Actual Output

Enter postfix expression: 23*54*+
Result: 26

Another Test Case

Input:
82/3-

Output:

Result: 1

Explanation:

8 / 2 = 4
4 - 3 = 1

PRACTICAL 4

Implement Tower of Hanoi using Stack

Problem Statement

Implement the **Tower of Hanoi problem using Stack**.

The Tower of Hanoi consists of three rods:

Source
Auxiliary
Destination

The objective is to move all disks from the source rod to the destination rod while following these rules:

1. Only one disk can be moved at a time.
2. Only the top disk can be removed.
3. A larger disk cannot be placed on a smaller disk.

Algorithm

For `n` disks:

1. Move `n-1` disks from Source to Auxiliary.
2. Move the largest disk from Source to Destination.
3. Move `n-1` disks from Auxiliary to Destination.
4. Repeat until all disks are moved.

For stack implementation:

1. Create three stacks representing the three rods.
2. Push disks onto the source stack in decreasing order.
3. Perform legal moves between the three stacks.
4. Continue until all disks reach the destination stack.

Input and Expected Output Test Case

Input

Number of disks = 3

Expected Output

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Source Code

```
#include <iostream>
using namespace std;

#define MAX 100

class Stack
{
public:
    int arr[MAX];
    int top;

    Stack()
    {
        top = -1;
    }

    void push(int value)
    {
        arr[++top] = value;
```

```
    }

    int pop()
    {
        return arr[top--];
    }

    bool empty()
    {
        return top == -1;
    }

    int peek()
    {
        return arr[top];
    }
};

void moveDisk(Stack &source, Stack &destination,
              char sourceName, char destinationName)
{
    if (source.empty())
    {
        int disk = destination.pop();
        source.push(disk);

        cout << "Move disk " << disk
             << " from " << destinationName
             << " to " << sourceName << endl;
    }
    else if (destination.empty())
    {
        int disk = source.pop();
        destination.push(disk);

        cout << "Move disk " << disk
             << " from " << sourceName
             << " to " << destinationName << endl;
    }
    else if (source.peek() < destination.peek())
    {
        int disk = source.pop();
        destination.push(disk);
```

```
        cout << "Move disk " << disk
             << " from " << sourceName
             << " to " << destinationName << endl;
    }
    else
    {
        int disk = destination.pop();
        source.push(disk);

        cout << "Move disk " << disk
             << " from " << destinationName
             << " to " << sourceName << endl;
    }
}

void towerOfHanoi(int n)
{
    Stack source, auxiliary, destination;

    for (int i = n; i >= 1; i--)
    {
        source.push(i);
    }

    int totalMoves = (1 << n) - 1;

    for (int i = 1; i <= totalMoves; i++)
    {
        if (n % 2 == 1)
        {
            if (i % 3 == 1)
                moveDisk(source, destination, 'A', 'C');
            else if (i % 3 == 2)
                moveDisk(source, auxiliary, 'A', 'B');
            else
                moveDisk(auxiliary, destination, 'B', 'C');
        }
        else
        {
            if (i % 3 == 1)
                moveDisk(source, auxiliary, 'A', 'B');
            else if (i % 3 == 2)
                moveDisk(source, destination, 'A', 'C');
            else
```

```
            moveDisk(auxiliary, destination, 'B', 'C');
        }
    }
}

int main()
{
    int n;

    cout << "Enter number of disks: ";
    cin >> n;

    towerOfHanoi(n);

    return 0;
}
```

Actual Output

Enter number of disks: 3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

PRACTICAL 5

Implement Queue and its Operations

Problem Statement

Implement a **Queue using an array** and perform the following operations:

- Creation
- Enqueue
- Dequeue

- Traverse
- Search

A queue follows the **FIFO (First In, First Out)** principle.

Algorithm

A. Enqueue

1. Check whether `rear == MAX - 1`.
2. If yes, display **Queue Overflow**.
3. Otherwise:
    - If the queue is empty, set `front = 0`.
    - Increment `rear`.
    - Insert the element at `queue[rear]`.

B. Dequeue

1. Check whether `front == -1` or `front > rear`.
2. If yes, display **Queue Underflow**.
3. Otherwise:
    - Delete `queue[front]`.
    - Increment `front`.

C. Traverse

1. Check whether the queue is empty.
2. Start from `front`.
3. Display elements up to `rear`.

D. Search

1. Read the element to search.
2. Start from `front`.
3. Compare each element.
4. If found, display its position.
5. Otherwise display **Element not found**.

Input and Expected Output Test Case

Input

Enqueue: 10
Enqueue: 20
Enqueue: 30
Enqueue: 40
Search: 30
Dequeue

Expected Output

Queue: 10 20 30 40
Element 30 found
Deleted element: 10
Queue: 20 30 40

Source Code

```
#include <iostream>
using namespace std;

#define MAX 100

int queueArr[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
   if (rear == MAX - 1)
   {
      cout << "Queue Overflow\n";
      return;
   }

   if (front == -1)
      front = 0;

   rear++;
   queueArr[rear] = value;

   cout << value << " inserted into queue.\n";
```

```
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue Underflow\n";
        return;
    }

    cout << "Deleted element: " << queueArr[front] << endl;

    front++;

    if (front > rear)
    {
        front = -1;
        rear = -1;
    }
}

void traverse()
{
    if (front == -1)
    {
        cout << "Queue is empty.\n";
        return;
    }

    cout << "Queue elements: ";

    for (int i = front; i <= rear; i++)
    {
        cout << queueArr[i] << " ";
    }

    cout << endl;
}

void search(int value)
{
    if (front == -1)
    {
        cout << "Queue is empty.\n";
```

```
        return;
    }

    for (int i = front; i <= rear; i++)
    {
        if (queueArr[i] == value)
        {
            cout << "Element " << value
                 << " found at position "
                 << i - front + 1 << endl;
            return;
        }
    }

    cout << "Element not found.\n";
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);

    traverse();

    search(30);

    dequeue();

    traverse();

    return 0;
}
```

Actual Output

```
10 inserted into queue.
20 inserted into queue.
30 inserted into queue.
40 inserted into queue.
Queue elements: 10 20 30 40
Element 30 found at position 3
```

Deleted element: 10
Queue elements: 20 30 40

PRACTICAL 6

Implement Singly Linked List and its Operations

Problem Statement

Implement a **Singly Linked List** and perform the following operations:

- Creation
- Insertion
- Deletion
- Traversal
- Search
- Reverse

Each node contains:

Data | Next

Algorithm

A. Creation

1. Create a new node.
2. Store data in the node.
3. Set `next = NULL`.
4. Attach the node at the end of the list.
5. Repeat for all elements.

B. Insertion at Beginning

1. Create a new node.
2. Store the data.
3. Set `newNode->next = head`.
4. Set `head = newNode`.

C. Insertion at End

1. Create a new node.
2. Set its `next = NULL`.
3. Traverse to the last node.
4. Set last node's `next` to the new node.

D. Deletion

1. Check whether the list is empty.
2. Search for the required element.
3. Adjust the links.
4. Delete the selected node.

E. Traversal

1. Start from `head`.
2. Display the data.
3. Move to the next node.
4. Continue until `NULL`.

F. Search

1. Start from `head`.
2. Compare each node's data with the search element.
3. If found, display the position.
4. Otherwise display **Element not found**.

G. Reverse

1. Initialize `previous = NULL`.
2. Initialize `current = head`.
3. Store the next node.
4. Reverse the current node's link.
5. Move `previous` and `current` forward.
6. Set `head = previous`.

Input and Expected Output Test Case

Input

Create: 10 20 30 40
Insert at beginning: 5
Insert at end: 50
Search: 30
Delete: 20
Reverse

Expected Output

Original List:
10 20 30 40

After insertion:
5 10 20 30 40 50

Element 30 found

After deletion:
5 10 30 40 50

After reverse:
50 40 30 10 5

Source Code

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *head = NULL;

void create(int value)
{
   Node *newNode = new Node;

   newNode->data = value;
   newNode->next = NULL;

   if (head == NULL)
```

```
    {
        head = newNode;
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}

void insertBeginning(int value)
{
    Node *newNode = new Node;

    newNode->data = value;
    newNode->next = head;

    head = newNode;
}

void insertEnd(int value)
{
    create(value);
}

void deleteNode(int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    if (head->data == value)
    {
        Node *temp = head;
        head = head->next;
        delete temp;
        return;
```

```
    }

    Node *temp = head;

    while (temp->next != NULL &&
           temp->next->data != value)
    {
        temp = temp->next;
    }

    if (temp->next == NULL)
    {
        cout << "Element not found.\n";
        return;
    }

    Node *deleteNode = temp->next;
    temp->next = deleteNode->next;

    delete deleteNode;
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

void search(int value)
{
    Node *temp = head;
```

```
    int position = 1;

    while (temp != NULL)
    {
        if (temp->data == value)
        {
            cout << "Element " << value
                 << " found at position "
                 << position << endl;
            return;
        }

        temp = temp->next;
        position++;
    }

    cout << "Element not found.\n";
}

void reverseList()
{
    Node *previous = NULL;
    Node *current = head;
    Node *nextNode;

    while (current != NULL)
    {
        nextNode = current->next;
        current->next = previous;

        previous = current;
        current = nextNode;
    }

    head = previous;
}

int main()
{
    create(10);
    create(20);
    create(30);
    create(40);
```

```
    cout << "Original List: ";
    traverse();

    insertBeginning(5);
    insertEnd(50);

    cout << "After insertion: ";
    traverse();

    search(30);

    deleteNode(20);

    cout << "After deletion: ";
    traverse();

    reverseList();

    cout << "After reverse: ";
    traverse();

    return 0;
}
```

Actual Output

```
Original List: 10 20 30 40
After insertion: 5 10 20 30 40 50
Element 30 found at position 4
After deletion: 5 10 30 40 50
After reverse: 50 40 30 10 5
```

PRACTICAL 7

Implement Doubly Linked List and its Operations

Problem Statement

Implement a **Doubly Linked List** and perform:

- Creation
- Insertion
- Deletion
- Traversal
- Search
- Reverse

Each node contains:

Previous | Data | Next

A doubly linked list allows traversal in both forward and backward directions.

Algorithm

A. Creation

1. Create a new node.
2. Store the data.
3. Set `prev = NULL` and `next = NULL`.
4. Attach the node at the end.
5. Update the `prev` link.

B. Insertion

1. Create a new node.
2. Insert it at the required position.
3. Update both `prev` and `next` links.

C. Deletion

1. Search for the required node.
2. Update the previous node's `next`.
3. Update the next node's `prev`.
4. Delete the node.

D. Traversal

1. Start from `head`.
2. Display each node.
3. Move using `next`.

E. Search

1. Start from `head`.
2. Compare each node's data.
3. If found, display its position.

F. Reverse

1. Traverse through every node.
2. Swap `prev` and `next`.
3. Move to the original previous node.
4. Update `head` to the last processed node.

Input and Expected Output Test Case

Input

Create: 10 20 30 40
Insert at beginning: 5
Insert at end: 50
Search: 30
Delete: 20
Reverse

Expected Output

Forward:
5 10 20 30 40 50

Element 30 found

After deletion:
5 10 30 40 50

After reverse:
50 40 30 10 5

Source Code

```
#include <iostream>
using namespace std;
```

```
struct Node
{
   int data;
   Node *prev;
   Node *next;
};

Node *head = NULL;

void create(int value)
{
   Node *newNode = new Node;

   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = NULL;

   if (head == NULL)
   {
      head = newNode;
      return;
   }

   Node *temp = head;

   while (temp->next != NULL)
   {
      temp = temp->next;
   }

   temp->next = newNode;
   newNode->prev = temp;
}

void insertBeginning(int value)
{
   Node *newNode = new Node;

   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = head;

   if (head != NULL)
```

```
    {
        head->prev = newNode;
    }

    head = newNode;
}

void insertEnd(int value)
{
    create(value);
}

void deleteNode(int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    while (temp != NULL && temp->data != value)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        cout << "Element not found.\n";
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    delete temp;
}
```

```
void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    cout << "Forward: ";

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

void search(int value)
{
    Node *temp = head;
    int position = 1;

    while (temp != NULL)
    {
        if (temp->data == value)
        {
            cout << "Element " << value
                 << " found at position "
                 << position << endl;
            return;
        }

        temp = temp->next;
        position++;
    }

    cout << "Element not found.\n";
}

void reverseList()
```

```
{
    Node *temp = head;
    Node *newHead = NULL;

    while (temp != NULL)
    {
        Node *nextNode = temp->next;

        temp->next = temp->prev;
        temp->prev = nextNode;

        newHead = temp;
        temp = nextNode;
    }

    head = newHead;
}

int main()
{
    create(10);
    create(20);
    create(30);
    create(40);

    cout << "Original List: ";
    traverse();

    insertBeginning(5);
    insertEnd(50);

    cout << "After insertion: ";
    traverse();

    search(30);

    deleteNode(20);

    cout << "After deletion: ";
    traverse();

    reverseList();

    cout << "After reverse: ";
```

```
    traverse();

    return 0;
}
```

Actual Output

```
Original List: Forward: 10 20 30 40
After insertion: Forward: 5 10 20 30 40 50
Element 30 found at position 4
After deletion: Forward: 5 10 30 40 50
After reverse: Forward: 50 40 30 10 5
```

PRACTICAL 8

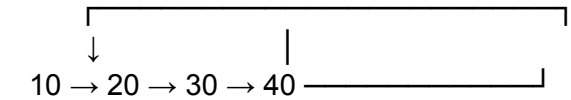
Implement Circular Linked List and its Operations

Problem Statement

Implement a **Circular Linked List** and perform:

- Creation
- Insertion
- Deletion
- Traversal
- Search
- Reverse

In a circular linked list, the last node points back to the first node.



Algorithm

A. Creation

1. Create a new node.
2. If the list is empty:
    - Make the new node the head.
    - Make its `next` point to itself.
3. Otherwise:
    - Traverse to the last node.
    - Connect the last node to the new node.
    - Connect the new node to `head`.

B. Insertion at Beginning

1. Create a new node.
2. Find the last node.
3. Set new node's `next = head`.
4. Set last node's `next = new node`.
5. Set `head = new node`.

C. Insertion at End

1. Create a new node.
2. Find the last node.
3. Set last node's `next = new node`.
4. Set new node's `next = head`.

D. Deletion

1. Search for the required node.
2. If it is the only node, set `head = NULL`.
3. If it is the first node, update the last node's link and `head`.
4. Otherwise, connect the previous node to the next node.
5. Delete the node.

E. Traversal

1. Start from `head`.
2. Display the current node.
3. Move to `next`.
4. Stop when the pointer reaches `head` again.

F. Search

1. Start from `head`.

2. Compare each node's data.
3. Stop when the pointer reaches `head`.
4. Display the position if found.

G. Reverse

1. Use three pointers: `previous`, `current`, and `nextNode`.
2. Reverse each node's link.
3. Continue until `current` reaches `head`.
4. Update the original head's link.
5. Set the new head.

Input and Expected Output Test Case

Input

Create: 10 20 30 40
Insert at beginning: 5
Insert at end: 50
Search: 30
Delete: 20
Reverse

Expected Output

Original List:
10 20 30 40

After insertion:
5 10 20 30 40 50

Element 30 found

After deletion:
5 10 30 40 50

After reverse:
50 40 30 10 5

Source Code

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *head = NULL;

void create(int value)
{
   Node *newNode = new Node;

   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
      newNode->next = head;
      return;
   }

   Node *temp = head;

   while (temp->next != head)
   {
      temp = temp->next;
   }

   temp->next = newNode;
   newNode->next = head;
}

void insertBeginning(int value)
{
   Node *newNode = new Node;

   newNode->data = value;

   if (head == NULL)
   {
      head = newNode;
```

```
        newNode->next = head;
        return;
    }

    Node *temp = head;

    while (temp->next != head)
    {
        temp = temp->next;
    }

    newNode->next = head;
    temp->next = newNode;
    head = newNode;
}

void insertEnd(int value)
{
    create(value);
}

void deleteNode(int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *current = head;
    Node *previous = NULL;

    // Only one node
    if (head->next == head)
    {
        if (head->data == value)
        {
            delete head;
            head = NULL;
        }
        else
        {
            cout << "Element not found.\n";
        }
```

```
        return;
    }

    // Delete first node
    if (head->data == value)
    {
        Node *last = head;

        while (last->next != head)
        {
            last = last->next;
        }

        last->next = head->next;

        Node *temp = head;
        head = head->next;

        delete temp;

        return;
    }

    previous = head;
    current = head->next;

    while (current != head)
    {
        if (current->data == value)
        {
            previous->next = current->next;
            delete current;
            return;
        }

        previous = current;
        current = current->next;
    }

    cout << "Element not found.\n";
}

void traverse()
```

```
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;

    do
    {
        cout << temp->data << " ";
        temp = temp->next;
    }
    while (temp != head);

    cout << endl;
}

void search(int value)
{
    if (head == NULL)
    {
        cout << "List is empty.\n";
        return;
    }

    Node *temp = head;
    int position = 1;

    do
    {
        if (temp->data == value)
        {
            cout << "Element " << value
                 << " found at position "
                 << position << endl;
            return;
        }

        temp = temp->next;
        position++;

    } while (temp != head);
```

```
    cout << "Element not found.\n";
}

void reverseList()
{
    if (head == NULL || head->next == head)
        return;

    Node *previous = NULL;
    Node *current = head;
    Node *nextNode;

    Node *oldHead = head;

    do
    {
        nextNode = current->next;
        current->next = previous;

        previous = current;
        current = nextNode;

    } while (current != head);

    oldHead->next = previous;
    head = previous;
}

int main()
{
    create(10);
    create(20);
    create(30);
    create(40);

    cout << "Original List: ";
    traverse();

    insertBeginning(5);
    insertEnd(50);

    cout << "After insertion: ";
    traverse();
```

```
    search(30);

    deleteNode(20);

    cout << "After deletion: ";
    traverse();

    reverseList();

    cout << "After reverse: ";
    traverse();

    return 0;
}
```

Actual Output

```
Original List: 10 20 30 40
After insertion: 5 10 20 30 40 50
Element 30 found at position 4
After deletion: 5 10 30 40 50
After reverse: 50 40 30 10 5
```
